Applied Artificial Intelligence
An International Journal

ISSN: 0883-9514 (Print) 1087-6545 (Online) Journal homepage: www.tandfonline.com/journals/uaai20

Evaluating LLMs for Code Generation in HRI: A Comparative Study of ChatGPT, Gemini, and Claude

Andrei Sobo, Awes Mubarak, Almas Baimagambetov & Nikolaos Polatidis

To cite this article: Andrei Sobo, Awes Mubarak, Almas Baimagambetov & Nikolaos Polatidis (2025) Evaluating LLMs for Code Generation in HRI: A Comparative Study of ChatGPT, Gemini, and Claude, Applied Artificial Intelligence, 39:1, 2439610, DOI: 10.1080/08839514.2024.2439610

To link to this article: https://doi.org/10.1080/08839514.2024.2439610

© 2024 The Author(s). Published with license by Taylor & Francis Group, LLC.

View supplementary material

Published online: 19 Dec 2024.

Submit your article to this journal

Article views: 6980

View related articles

View Crossmark data

Citing articles: 14 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=uaai20

APPLIED ARTIFICIAL INTELLIGENCE
2025, VOL. 39, NO. 1, e2439610 (22 pages)
https://doi.org/10.1080/08839514.2024.2439610

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS Check for updates

Evaluating LLMs for Code Generation in HRI: A Comparative Study of ChatGPT, Gemini, and Claude

Andrei Sobo, Awes Mubarak, Almas Baimagambetov, and Nikolaos Polatidis

School of Architecture, Technology and Engineering, University of Brighton, Brighton, UK

ABSTRACT

This study investigates the effectiveness of Large Language Models (LLMs) in generating code for Human-Robot Interaction (HRI) applications. We present the first direct comparison of ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet in the specific context of generating code for Human-Robot Interaction applications. Through a series of 20 carefully designed prompts, ranging from simple movement commands to complex object manipulation scenarios, we evaluate the models' ability to generate accurate and context-aware code. Our findings reveal significant variations in performance, with Claude 3.5 Sonnet achieving a 95% success rate, Gemini 1.5 Pro at 60%, and ChatGPT 3.5 at 20%. The study highlights the rapid advancement in LLM capabilities for specialized programming tasks while also identifying persistent challenges in spatial reasoning and adherence to specific constraints. These results suggest promising applications for LLMs in robotics development and education while emphasizing the continued need for human oversight and specialized training in AI-assisted programming for HRI.

ARTICLE HISTORY
Received 21 September 2024
Revised 16 November 2024
Accepted 27 November 2024

Introduction

The rapid advancement of Large Language Models (LLMs) has opened new frontiers in various domains, including Human-Robot Interaction (HRI). As robots become increasingly integrated into human environments, from manufacturing floors to healthcare facilities and homes, the need for efficient and accurate programming of these systems becomes paramount (Sheridan 2016). The potential of LLMs to generate code based on natural language inputs presents a promising avenue for streamlining the development process in HRI applications (Chen et al. 2021; Li et al. 2022).

However, the application of LLMs in generating code for robotic systems presents unique challenges. Unlike general-purpose programming tasks, code generation for HRI requires a nuanced understanding of spatial relationships, physical constraints, and the ability to translate abstract concepts into precise

CONTACT Nikolaos Polatidis, N.Polatidis@brighton.ac.uk School of Architecture, Technology and Engineering, University of Brighton, Cockcroft 521, Moulsecoomb campus, Brighton BN2 4GJ, UK

© 2024 The Author(s). Published with license by Taylor & Francis Group, LLC.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

movement instructions (Zeng et al. 2023). The complexity of these tasks raises questions about the current capabilities of LLMs in this specialized domain.

This study aims to address a gap in the current literature by providing a comparative analysis of state-of-the-art LLMs in generating code for an abstract robot class. Specifically, we investigate the performance of ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet in translating natural language instructions into functional Java code for robotic movement and object manipulation tasks. Our research is guided by the following questions:

(1) How accurately can current LLMs generate code for basic and complex robotic movement patterns based on natural language instructions?
(2) To what extent do these models demonstrate understanding of spatial relationships and physical constraints in a three-dimensional environment?
(3) How do the performance and capabilities of ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet compare in the context of HRI code generation?
(4) What are the current limitations and areas for improvement in using LLMs for robotic programming tasks?

By addressing these questions, our study contributes to the growing body of research on AI-assisted programming in robotics (Xu et al. 2022).; Crucially, this study addresses a significant gap in the current literature by providing the first direct comparison of ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet in the specific context of robotics code generation. While previous studies have examined LLMs in general code generation tasks, our focus on these state-of-the-art models in the specialized domain of HRI programming represents a novel contribution to the field. Furthermore, while traditional software engineering often employs code complexity metrics to evaluate code quality, our study deliberately focuses on a pass/fail approach. This decision reflects the unique requirements of Human-Robot Interaction programming, where the primary concern is the correct execution of physical tasks rather than the internal complexity of the code. By prioritizing functionality over traditional metrics, we aim to provide insights that are directly applicable to real-world HRI scenarios, where the consequences of code failure can be immediate and physical. This approach also allows us to more clearly differentiate between the LLMs' abilities to generate code that accurately translates natural language instructions into executable robot commands.

Moreover, this research provides insights into the broader question of how well LLMs can bridge the gap between natural language understanding and the generation of precise, executable instructions for physical systems (Pearce et al. 2022). By comparing these specific LLMs, which represent different approaches and recent advancements in language model development, our

study offers a unique perspective on the current capabilities and limitations of AI in translating human intent into formal, executable code for complex, real-world robotic applications. This comparison not only highlights the rapid progress in LLM technology but also identifies areas where further development is needed to fully realize the potential of AI-assisted programming in robotics.

While this study provides valuable insights into the capabilities of LLMs in robotics programming, it is important to acknowledge its limitations. The study focuses specifically on code generation for an abstract robot class, which, while representative of many robotic systems, does not capture the full complexity of real-world robotics programming. The use of a discrete, integer-based coordinate system and simplified object interaction model allows for clear evaluation criteria but may not fully reflect the continuous nature of physical space or the intricacies of real-world object manipulation. Additionally, the study is limited to Java programming language and does not explore performance across different programming paradigms. This narrow focus was chosen to provide a controlled environment for comparison, allowing for precise evaluation of the LLMs' capabilities in translating natural language instructions into executable code for robotic control. By constraining the scope, we aim to provide a clear benchmark for current LLM performance in this specific domain, which can serve as a foundation for future, more expansive studies.

Background

Evolution of large language models

The development of LLMs represents a significant paradigm shift in NLP, marking a transition from task-specific models to more generalized, multi-purpose systems (Bommasani et al. 2021). This evolution has been characterized by exponential increases in model size, architectural innovations, and novel training approaches, leading to unprecedented capabilities in language understanding and generation (Brown 2020).

The foundation for modern LLMs was laid with the introduction of the Transformer architecture by (Vaswani 2017), which enabled more efficient processing of sequential data and facilitated the training of larger models. This architectural innovation was quickly adopted and refined, leading to the development of models like BERT (Bidirectional Encoder Representations from Transformers), which demonstrated superior performance across a wide range of NLP tasks.

The trend towards larger model sizes gained momentum with the introduction of GPT (Generative Pre-trained Transformer) models by OpenAI. GPT-2, released in 2019, showcased the potential of large-scale language models with

1.5 billion parameters (Radford et al. 2019). This was followed by the landmark GPT-3 model in 2020, which, with its 175 billion parameters, demonstrated remarkable few-shot learning capabilities across diverse tasks (Brown 2020).

The success of these models sparked a race to develop ever-larger LLMs, with companies and research institutions investing heavily in this technology. Models like Megatron-Turing NLG (530 billion parameters) and PaLM (540 billion parameters) pushed the boundaries of model size and performance (Smith et al. 2022).

In response to concerns about the computational resources required for training and deploying such large models, recent research has focused on developing more efficient training methods and model architectures. Techniques such as sparse attention mechanisms (Child et al. 2019), mixture-of-experts architectures, and retrieval-augmented generation (Lewis et al. 2020) have emerged as promising approaches to improve model efficiency without sacrificing performance.

Another significant development in the evolution of LLMs has been the shift toward instruction-tuning and multi-task training. Models like InstructGPT and T5 have demonstrated the benefits of fine-tuning LLMs on diverse sets of tasks framed as instructions, leading to improved performance and generalization capabilities (Ouyang et al. 2022; Raffel et al. 2020).

Specific LLM philosophies and technologies: ChatGPT, Gemini, and Claude

The three LLMs evaluated in this study – ChatGPT, Gemini, and Claude – represent different approaches and philosophies in the development of large language models.

ChatGPT, developed by OpenAI, is based on the GPT (Generative Pre-trained Transformer) architecture. It utilizes a technique called Reinforcement Learning from Human Feedback (RLHF) to align the model's outputs with human preferences (Bai et al. 2022; Ouyang et al. 2022). This approach involves fine-tuning the model on a curated dataset of human-written responses, followed by further optimization using reward modeling and reinforcement learning (Bai et al. 2022).

Gemini, developed by Google, represents a departure from the traditional transformer-based architecture. While specific details about Gemini's architecture are not publicly available, Google has stated that it incorporates novel attention mechanisms and more efficient training techniques. Gemini is designed as a multimodal model, capable of processing and generating not just text, but also images, audio, and video.

Claude, developed by Anthropic, is built on the philosophy of constitutional AI, aiming to create AI systems that are more aligned with human values and ethical considerations (Askell et al. 2021). While the exact details of Claude's

training process are not public, it is known to incorporate techniques for improving truthfulness and reducing harmful outputs (Bai et al. 2022). Claude has demonstrated strong performance in tasks requiring logical reasoning and adherence to specific instructions, making it potentially well suited for code generation tasks in specialized domains like robotics (Perez et al. 2022).

Code generation capabilities of LLMs

The ability of LLMs to generate code represents an emergent property that was not explicitly programmed into these systems. This capability arises from the fundamental architecture and training process of LLMs, which allows them to capture and reproduce complex patterns in various forms of text, including programming languages (Brown 2020; Chen et al. 2021).

At their core, LLMs are trained on vast corpora of text data, which often include substantial amounts of code and programming-related discussions (Husain et al. 2019; Xu et al. 2022). This exposure allows the models to learn the patterns, structures, and conventions of various programming languages implicitly, without being explicitly taught the rules of syntax or semantics. The self-attention mechanisms in transformer-based models, which form the backbone of modern LLMs, enable them to capture long-range dependencies in text (Vaswani 2017). This characteristic is particularly crucial for understanding and generating code, as programming languages often involve complex nested structures and interdependencies between different parts of the code.

The process by which LLMs generate code can be understood as a form of statistical pattern matching and completion. When presented with a prompt or a partial code snippet, the model draws upon its learned representations to predict the most likely continuations based on the patterns it has observed in its training data (Austin et al. 2021). This process is similar to how LLMs generate natural language, but with the added complexity of adhering to the strict syntactical rules of programming languages.

Interestingly, the ability of LLMs to generate code emerges as a byproduct of their general language understanding capabilities (Wei et al. 2022). As these models become more sophisticated in their ability to understand and generate natural language, they simultaneously develop the capacity to work with formal languages like programming code.

Introduction to coding in HRI and challenges

Human-Robot Interaction (HRI) has emerged as a critical field at the intersection of robotics, computer science, and cognitive psychology. As robots increasingly integrate into various aspects of human life, from manufacturing and healthcare to domestic assistance and space exploration, the need for

effective and intuitive interaction between humans and robots has become paramount (Sheridan 2016). This interaction relies heavily on sophisticated programming that enables robots to interpret human commands, respond appropriately to their environment, and operate safely alongside humans.

Programming plays a central role in HRI, serving as the foundation for a robot's ability to perceive, process, and respond to human input and environmental stimuli. It encompasses a wide range of functionalities, including natural language processing for verbal communication, computer vision for interpreting visual cues and gestures, motion planning for navigation and manipulation tasks, and decision-making algorithms for autonomous behavior (Breazeal, Dautenhahn, and Kanda 2016).;

However, programming for HRI presents unique challenges that set it apart from traditional software development. First, HRI systems must operate in dynamic, unpredictable environments where they interact with humans who may have varying levels of familiarity with robotic systems (Hoffman and Zhao 2020). This requires robust error handling, adaptive behavior, and the ability to interpret ambiguous or incomplete instructions. Second, HRI programming must account for the physical embodiment of robots, considering factors such as spatial awareness, motor control, and real-time responsiveness. Third, there are critical safety and ethical considerations in HRI programming, as errors or misinterpretations can lead to physical harm or unintended consequences (Winfield and Jirotka 2018). Finally, HRI programming often requires interdisciplinary knowledge, combining expertise in robotics, psychology, and human factors engineering to create truly effective interaction paradigms.

The advent of advanced code generation techniques, particularly those leveraging LLMs, holds significant potential for addressing these challenges in HRI programming. Of particular interest is the application of LLMs to generate code for controlling robot movement patterns, a fundamental component in many HRI systems. This approach involves translating natural language instructions (e.g., "move from point A to point B") into executable code that manipulates the robot's state and position (Li et al. 2022). The ability to generate such code efficiently and accurately could dramatically streamline the development of HRI systems, allowing researchers and developers to focus on higher-level interaction design rather than low-level implementation details (Chen et al. 2021).

This application of LLMs to robot movement code generation raises profound questions about the nature of programming in HRI and the potential for AI-assisted development. It touches on fundamental issues in cognitive science and AI, such as the relationship between natural language understanding and spatial reasoning, the abstraction of physical concepts in programming languages, and the potential for machines to bridge the gap between

human intent and formal executable instructions (Xu et al. 2022). Moreover, it provides a unique lens through which to examine the capabilities and limitations of LLMs in understanding and generating code for physical systems, potentially offering insights into both the nature of machine learning and the essence of human-robot interaction itself.

Related work

The application of LLMs to code generation, particularly in the context of robotics and HRI, has gained significant attention in recent years. This literature review examines the current state of research in this field, focusing on the capabilities and limitations of LLMs in generating code for robotic systems, the challenges in translating natural language to robot control code, and the existing comparative studies and evaluation frameworks.

LLMs have demonstrated remarkable capabilities in code generation across various programming tasks including the interaction with robots (Zhang et al. 2023). Chen et al. (2021) conducted a comprehensive evaluation of LLMs trained on code, highlighting their potential in assisting software development. Their study revealed that LLMs can generate functionally correct code for a significant portion of programming problems, though performance varies across task complexity. Similarly, Li et al. (2022) showcased the advanced capabilities of LLMs in generating complex code through their work on AlphaCode, demonstrating competition-level performance. However, Xu et al. (2022) provided a more nuanced view, systematically evaluating what LLMs truly understand about programming. Their findings suggest that while LLMs excel in syntax and common coding patterns, they may struggle with deeper logical reasoning and domain-specific knowledge.

In the realm of robotics and HRI, LLMs have shown promise in bridging the gap between natural language and robot control. Kwon, Di Palo, and Johns (2024) demonstrated the use of LLMs as zero-shot task planners for robotic manipulation, illustrating how these models can interpret high-level commands and generate appropriate action sequences. This work, however, focused on task planning rather than direct code generation. Ahn et al. (2022) explored grounding language in robotic affordances, addressing the crucial issue of aligning natural language instructions with a robot's physical capabilities. Their approach, while not directly generating code, provides insights into how LLMs might be used to interpret and execute human instructions in robotic contexts.

The translation of natural language to robot control code represents a significant challenge and opportunity in this field. Zeng et al. (2023) provided a comprehensive survey on natural language robot programming, highlighting various approaches and ongoing challenges. Their work underscores the potential of LLMs in this domain while also noting the

complexities involved in translating human intent into precise robot actions. Song et al. (2023) introduced LLM-Planner, demonstrating how LLMs can be used for few-shot grounded planning in embodied agents. While their focus was on high-level planning rather than low-level control code, their work illustrates the potential of LLMs in generating structured outputs for robotic systems.

Comparative studies of LLMs in robotics applications are relatively scarce, representing a significant gap in the literature. While there have been general comparisons of LLMs in code generation tasks, such as the work by Xu et al. (2022), these studies typically do not focus specifically on robotics applications or the generation of control code for abstract robot classes. The lack of direct comparisons between state-of-the-art LLMs like ChatGPT, Gemini, and Claude in the context of robot code generation represents a notable gap in current research.

The challenges and evaluation frameworks for LLM-generated code in robotics contexts present another area requiring further exploration. Pearce et al. (2022) examined the security implications of AI-generated code, highlighting potential risks that could be particularly critical in robotic applications. However, there is a lack of standardized evaluation metrics specifically tailored to assessing LLM-generated code for robotic control, particularly in terms of safety, efficiency, and adherence to physical constraints. This gap in evaluation frameworks is particularly notable given the unique requirements of robotic systems, where code errors could have significant real-world consequences. The development of robust, domain-specific evaluation methods for LLM-generated robotic code remains an important area for future research.

Ba¨rmann et al. (2024) in their work used natural language to communicate with a robot and to improve commands if the robot doesn't understand exactly what is required to do. They use Python and LLMs to generate code that can be executed in robots while they use incremental learning to help robots become better in understanding human instructions. Furthermore, there are more recent works as well where LLMs are used to interact with robots. Wang et al. (2024) are using LLMs for multi modal human-robot interaction. This is still a work in progress but one that manages successfully to create atomic actions to instruct robots to perform simple actions such as moving the neck or ears. Yet in another work from Park, Menassa, and Kamat (2025), the authors use LLMs with multi modal virtual reality to support collaborative work in construction environments. Moreover, another similar work is the one from Izquierdo-Badiola et al. (2024) where the authors are leveraging LLMs for developing adaptive plan generation in human-robot interaction. Last but not least, is the work from Mahadevan et al. (2024) where the authors use LLMs for generative expressive robot behaviors that build up on each other.

In conclusion, while significant progress has been made in applying LLMs to code generation and robotic task planning, several key gaps remain. These include the lack of comprehensive comparisons between state-of-the-art LLMs in generating code for abstract robot control, the need for specialized evaluation frameworks for LLM-generated robotic code, and the challenges in translating natural language commands into precise, executable code for robot movement in 3D space. Additionally, the application of LLMs to generate code for abstract robot classes, which combines natural language processing, spatial reasoning, and code generation, remains underexplored. These gaps highlight the need for further research in this area, particularly studies that directly compare the performance of different LLMs in generating code for robotic movement based on natural language inputs. Moreover, to address the question of achieving more generic results, we acknowledge that our study focuses on a specific application of LLMs for controlling robotic systems. However, the core methodologies and findings could indeed be adapted to other tools and programming languages with adjustments. To this extent, we deliver a framework that can be easily modified to work with other tools. While our initial implementation uses a specific set of tools, the principles guiding the interaction between LLMs and robotic systems could be extended to similar architectures by adapting interfaces. Moreover, while our results are based on one application, they could be verified across different languages and platforms by adjusting the code generation, language parsing, and robotic control mechanisms. We are considering future experiments with different programming environments (e.g., Python, Java, or C++), which would allow us to test reproducibility and enhance the framework's versatility. Finally, our current results serve as a foundation for developing protocols that could enable other researchers to replicate our level of achievement. For example, by open-sourcing our code and methods, we hope to invite the research community to experiment and verify our approach across diverse programming tools and languages. By these means, we believe our work can contribute toward a generalized approach for using LLMs in robotics, allowing broader applications and easing reproduction efforts across platforms.

Methodology

This study employed a comprehensive testing approach to evaluate the performance of three LLMs – ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet – in generating code for an abstract robot class. The methodology combined automated unit testing with a carefully designed set of prompts to assess both the functionality and accuracy of the generated code.

We evaluated the models using a high-level abstraction, which can be implemented using domain-specific languages. For instance, instructions such as "moveX" can be implemented in programming languages like C++,

Python, or others. Although Java is used as an example of a high-level language with an established ecosystem for setting up testing environments, it is not a strict requirement for the implementation.

Test environment design

The foundation of our study rests upon a test environment centered around an abstract Robot class. This class serves as a theoretical model of a robotic system capable of movement in three-dimensional space and object manipulation. The Robot class operates within a discrete, integer-based coordinate system, allowing for precise control and measurement of its actions. Figure 1 gives a high-level overview of the test environment.

At the core of the Robot class is its position, represented by three integer variables: x, y, and z. The robot's default starting position is set at coordinates (0, 2, 0), chosen to simulate a realistic scenario where the robot begins slightly elevated above the ground plane. This design choice allows for more complex movements, including the ability to lower the robot's position when interacting with objects on the ground.

Movement capabilities are implemented through three primary methods: moveX(), moveY(), and moveZ(). Each of these methods accepts an integer parameter representing the distance and direction of movement along its respective axis. Positive values indicate movement in the positive direction of the axis, while negative values represent movement in the negative direction.

A critical component of the Robot class is its gripper mechanism, represented by a boolean variable gripperIsOpen. This variable simulates the robot's ability to grasp and release objects. The gripper's state is toggled using openGripper() and closeGripper() methods, providing a simple yet effective model of object interaction.

To test for realistic behavior we implemented a collision detection system through the checkCollision() method. This method is automatically called before any movement operation and during gripper actions. It utilizes the robot's current position, the proposed new position, and the state of the gripper to determine if a collision would occur. If the gripper is closed and the robot attempts to move to a position occupied by an object, the method throws an IllegalStateException, throwing impossible movements.

Objects within the robot's environment are represented by instances of the ObjectHRI class. This class mirrors the positional attributes of the Robot class, containing x, y, and z coordinates. Objects can be added to the robot's environment, picked up, moved, and placed, allowing for complex interaction scenarios.

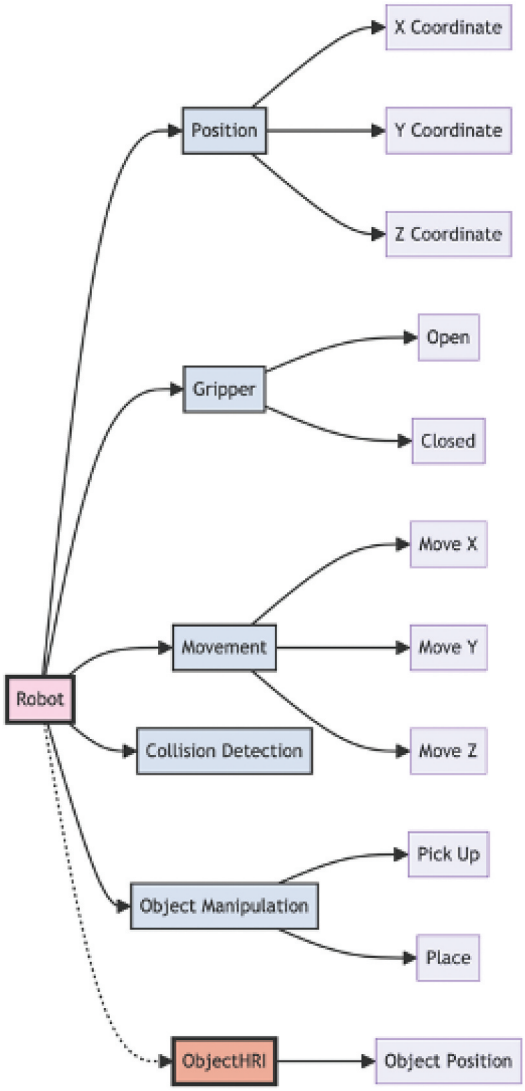

Figure 1. Test environment design.

The Robot class includes several higher-level methods that combine basic movements and gripper operations to perform more complex tasks. These methods, such as pickupObjectAt3_0_9() or moveAndPlaceObjectForward(), were designed to correspond directly to the prompts given to the LLMs, providing a clear way to evaluate the generated code's effectiveness.

To allow for testing and position tracking, we implemented getPosition() methods in both the Robot and ObjectHRI classes. These methods return an ArrayList!Integer¿ containing the current x, y, and z coordinates, allowing for comparison between expected and actual positions after operations.

While this abstract test environment doesn't account for all real-world factors such as momentum, friction, or continuous space, it provides a sufficiently complex system to challenge the LLMs' ability to generate meaningful and correct code for robotic operations. The discrete, integer-based coordinate system simplifies position tracking and collision detection while still allowing for a wide range of movement and manipulation tasks.

Prompt design

The cornerstone of our study was a carefully curated set of 20 prompts designed to challenge the LLM in generating code for robotic tasks. These prompts were crafted to simulate a range of scenarios a robotic system might encounter, varying in complexity and spatial reasoning requirements. The design process focused on creating a comprehensive test suite that would evaluate the LLMs' ability to translate natural language instructions into functional code for our abstract Robot class and reflect real-world considerations with regards to HRI.

Each prompt was formulated as a clear, concise instruction that a human programmer might receive when tasked with controlling a robotic system. The language used was intentionally colloquial, mimicking real-world scenarios where technical and non-technical stakeholders might provide requirements for robotic operations. This approach allowed us to assess not only the LLMs' code generation capabilities but also their ability to interpret and contextualize natural language within the domain of robotics.

The prompts were strategically designed to cover a spectrum of robotic operations, including:

(1) **Basic Movement**: Simple instructions to move the robot to specific coordinates, testing the LLMs' understanding of the robot's movement methods. For example, "pickup object at (3,0,9). Do not return to origin" requires basic movement along two axes and introduces a constraint on the robot's final position.
(2) **Relative Positioning**: Tasks involving movements relative to the robot's current position or the origin, such as "pickup object that is 7 cm to the left and 4 cm forward of origin." These prompts assess the LLMs' ability to translate relative spatial descriptions into absolute coordinate movements.
(3) **Object Manipulation**: Instructions to pick up, move, and place objects, testing the LLMs' understanding of the robot's gripper functionality and object interaction. An example is "pickup the object situated 2 cm to the right from origin and move it at coordinates (−2,0,2)."
(4) **Multi-step Operations**: Complex sequences involving multiple movements and object interactions, like "Take the object at (1,0,1) and place

it at (2,0,1). Then, take another object at (3,0,3) and place it one cm to the right." These prompts evaluate the LLMs' ability to plan and execute a series of interdependent actions.

(5) **Spatial Reasoning**: Tasks requiring interpretation of spatial relationships, such as "There are 3 objects. First is at coordinates (3,0,1), second is at (4,0,2) and third at (−2,0,2). I want you to pick them and place them on top of each other at coordinates (−5,0,5)." This tests the LLMs' capability to manage multiple objects in three-dimensional space.

(6) **Constraint Handling**: Prompts with specific constraints or conditions, like "do not lift the arm up after placing" or "do not return to origin." These assess the LLMs' attention to detail and ability to adhere to operational constraints.

(7) **Abstract Concepts**: Some prompts introduced more abstract concepts, such as "From origin, draw a square in the air with a side length of 3," testing the LLMs' ability to translate conceptual ideas into concrete movement patterns.

The sequence of prompts was carefully ordered to gradually increase in complexity. This progressive difficulty allowed us to assess how the LLMs' performance changed as tasks became more intricate, providing insights into their limitations and strengths. Each prompt was designed to have a clear, objectively verifiable outcome. To ensure consistency across all three LLMs (ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet), the exact same set of prompts was used in the same order for each model.

Testing process

The testing process began with the presentation of each of the 20 prompts to the LLMs. We maintained a consistent format for these presentations, instructing each model to generate Java code that would implement the requested functionality using our abstract Robot class. Crucially, each LLM was given only one attempt per prompt, mirroring real-world scenarios where immediate, effective code generation is often necessary.

Upon receiving the generated code from an LLM, we proceeded with an integration process. This involved incorporating the generated method into our Robot class within the test environment. During this stage, we ensured that the code was properly formatted and syntactically correct to allow for compilation. It is worth noting that while we corrected minor syntax errors such as missing semicolons, we strictly adhered to a policy of not altering any logical or algorithmic aspects of the generated code. This preservation of the LLM's original output was crucial for maintaining the integrity and fairness of our evaluation.

The core of our testing process lay in the execution of a comprehensive suite of unit tests, developed using the JUnit framework. Each of the 20 prompts had a corresponding unit test method in our RobotTest class, designed to verify whether the robot and any manipulated objects reached their correct final positions after executing the generated code. These tests typically involved initializing a Robot instance and any necessary ObjectHRI instances, executing the LLM-generated method, and then using assertions to verify the final positions and states.

To illustrate, consider the test for the prompt "pickup object at (3,0,9). Do not return to origin." This test initialized a Robot, called the generated method, and then verified that both the robot and the object ended up at the correct coordinates. These tests allowed us to objectively assess whether the generated code accurately interpreted and executed the given instructions.

As we executed the complete test suite for each LLM, we meticulously recorded the outcomes. For each test, we noted whether it passed or failed, any exceptions or errors that occurred during execution, and the final positions of the robot and objects. This detailed data collection allowed us to build a comprehensive picture of each LLM's performance.

Our analysis of the collected data was objective. The test classes examined each LLM's overall success rate, their performance on tasks of varying complexity, their ability to handle spatial reasoning challenges, and their adherence to specific constraints mentioned in the prompts. We paid particular attention to how well each model implemented the collision detection system, as this aspect is crucial for safe and logical robotic operations.

Throughout the testing process, we maintained a strong focus on reproducibility and consistency. We used the same version of Java, the same JUnit framework, and identical hardware for running tests across all three LLMs. Our use of version control for the test code ensured that we could track any changes and maintain consistency throughout the study.

The design of our test environment is informed by prior research in both robotics testing and the application of LLMs in control and automation tasks. While studies directly focused on using LLMs for robotics are indeed scarce, we drew insights from established methodologies in related fields to ensure robustness. Moreover, our work involves a unique integration of LLMs with robotic systems, we implemented custom test scenarios designed to reflect real-world robotic challenges – such as object manipulation, navigation, and error recovery. This approach allows us to test not only the direct outputs of the LLM but also how these outputs translate into physical actions, validating both the language model's accuracy and the environment's setup.

Furthermore, regarding the design and origin of the prompts, it is worth mentioning that the prompts are inspired by previous studies that explore effective ways of instructing LLMs to perform specific tasks. Additionally, we

looked into prompt engineering techniques that help tailor the LLM's responses to match the required robotic operations, ensuring each command reliably invokes the desired action. To this extent, to invoke the operations, each prompt is structured with specific, unambiguous language that the LLM can interpret as a command for a corresponding robotic task.

Results

Claude 3.5 Sonnet exhibited exceptional performance, successfully completing 19 out of 20 tasks, achieving a 95% success rate. This model demonstrated a robust understanding of spatial relationships, precise object manipulation, and strict adherence to given instructions. It consistently managed object states correctly, updating object coordinates appropriately after movements. Claude 3.5 Sonnet's generated code was well structured and efficient, often including helper methods for repeated operations. Its performance remained high across both simple and complex tasks, including multi-step operations involving multiple objects.

Gemini 1.5 Pro showed a marked improvement over ChatGPT 3.5, successfully completing 12 out of 20 tasks, resulting in a 60% success rate. This model performed well on simpler tasks but showed decreased accuracy as task complexity increased. Gemini 1.5 Pro demonstrated better spatial reasoning capabilities than ChatGPT 3.5 but still encountered issues with object collisions in more complex scenarios. The code generated by Gemini 1.5 Pro was generally well structured though it occasionally contained redundancies. While it showed improvements in error handling compared to ChatGPT 3.5, there were still instances where potential errors were not accounted for.

ChatGPT 3.5 struggled significantly with the given tasks, passing only 4 out of 20 tests, resulting in a 20% success rate. The model frequently failed to adhere to crucial instructions, such as returning to the origin after completing tasks – a mistake that accounted for 12 out of its 16 failures. ChatGPT 3.5 demonstrated significant difficulties with spatial reasoning, often making errors in calculating correct movements along different axes. It frequently mismanaged object states and gripper operations, leading to potential collisions or dropping objects. The model struggled even with simpler tasks and often failed to complete all required steps in more complex scenarios. The code generated by ChatGPT 3.5 often included unnecessary movements or inefficient sequences of operations, and frequently lacked proper error handling, particularly in scenarios involving potential object collisions.

Claude 3.5 Sonnet's performance suggests a significant advancement in understanding and implementing complex instructions, maintaining context, and handling spatial reasoning in a robotics environment. Gemini 1.5 Pro's improvements over ChatGPT 3.5 indicate the ongoing evolution of these models. However, ChatGPT 3.5's struggles underscore the challenges that

earlier models faced in maintaining context and following multi-step instructions in specialized domains like robotics.

Figure 2 presents the results.

The results clearly demonstrate the superiority of Claude 3.5 Sonnet in robotic task execution, with an impressive 95% success rate across both simple and complex tasks. Its robust handling of spatial relationships, accurate object manipulation, and efficient code generation underline its advanced capabilities compared to its predecessors. The model's ability to successfully manage multi-step tasks and maintain object state integrity showcases its suitability for real-world robotic applications, where precision and adherence to instructions are crucial. This performance indicates that Claude 3.5 Sonnet has the potential to be integrated into more complex and dynamic environments, where it can effectively manage both simple and advanced tasks without significant issues.

In contrast, the findings from Gemini 1.5 Pro and ChatGPT 3.5 highlight the challenges and ongoing improvements in the field of generative AI for robotics. While Gemini 1.5 Pro outperformed ChatGPT 3.5 with a 60% success rate, its struggle with more complex tasks and occasional code redundancies suggest that it is still refining its spatial reasoning and task handling. Despite improvements in error management, the model's failure to prevent object collisions in some scenarios indicates room for growth, particularly in high-stakes operations where precision is key. ChatGPT 3.5's limited success (20%) further emphasizes the difficulty earlier models have in executing robotics-related tasks, especially in following multi-step instructions and ensuring safe object handling. These results reinforce the need for further model refinement, especially in handling complex, real-world robotic challenges, and underline the importance of incorporating more specialized training to improve reliability and task accuracy.

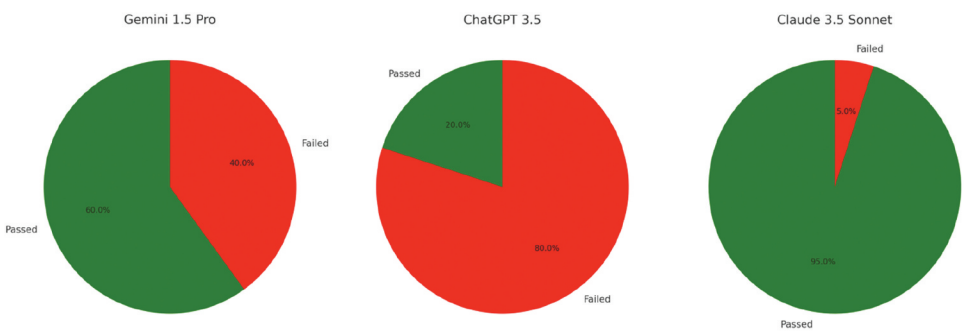

Figure 2. Performance comparison of ChatGPT 3.5, Gemini 1.5 pro, and Claude 3.5 Sonnet.

Discussion

The results of our comparative study on ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet in generating code for an abstract robot class reveal insights into the current capabilities and limitations of LLMs in the context of HRI programming.

Claude 3.5 Sonnet's exceptional performance, with a 95% success rate, demonstrates the rapid advancements in LLM technology. Its ability to consistently generate well-structured, efficient code while maintaining spatial awareness and adhering to complex instructions suggests that state-of-the-art LLMs are approaching a level where they could be reliably used in robotics programming tasks. The inclusion of helper methods for repeated operations in Claude's generated code indicates a sophisticated understanding of programming best practices, which could potentially enhance code maintainability and readability in real-world applications.

Gemini 1.5 Pro's performance, while not as impressive as Claude's, still shows significant improvement over ChatGPT 3.5. Its 60% success rate and better handling of spatial reasoning tasks suggest that LLMs are evolving rapidly in their ability to understand and implement domain-specific concepts. However, the issues encountered with object collisions in more complex scenarios highlight the ongoing challenges in translating natural language instructions into precise, error-free code for robotic operations.

ChatGPT 3.5's struggles, particularly with spatial reasoning and adherence to crucial instructions, underscore the complexity of robotics programming and the importance of continued development in LLM training for specialized domains. The high failure rate in returning the robot to its origin point after task completion (accounting for 75% of its failures) emphasizes the need for LLMs to maintain context and follow multi-step instructions more reliably in specialized applications.

These findings have several implications for the field of HRI and AI-assisted programming:

(1) **LLM Integration in Robotics Development**: As LLMs like Claude 3.5 Sonnet demonstrate high proficiency in generating accurate robotics code, we may see increased integration of these models in robotics development workflows. This could potentially accelerate prototyping and reduce the initial coding burden on human programmers.
(2) **Specialised Training for Robotics LLMs**: The performance disparity between the models suggests that specialized training focused on spatial reasoning, physical constraints, and robotics-specific concepts could significantly enhance LLMs' capabilities in this domain.

(3) **Human-AI Collaboration**: While the results are promising, the imperfect performance even of the best model (Claude 3.5 Sonnet) indicates that human oversight remains crucial.

Future research directions should focus on several key areas:

(1) **Longitudinal Studies**: As LLM technology is rapidly evolving, conducting regular comparative studies would provide valuable insights into the pace of improvement and emerging capabilities in robotics code generation.
(2) **Language Diversity**: Our study focused on Java, but future research should explore the performance of LLMs across various programming languages. This could reveal whether the abundance of training data in popular languages like Python or Java significantly impacts LLM performance compared to less common languages.
(3) **Complex Scenarios**: Developing more sophisticated test cases that involve advanced robotics concepts, such as sensor integration, real-time decision-making, and multi-robot coordination, would further push the boundaries of LLM capabilities in this domain.
(4) **Fine-tuning for Robotics**: Investigating the impact of fine-tuning LLMs on large datasets of robotics-specific code could potentially enhance their performance in this specialized field.

While the results demonstrate significant progress in LLMs' ability to generate code for HRI applications, they also highlight areas for improvement. The rapid advancement evident in the performance difference between models underscores the dynamic nature of this field and the potential for further breakthroughs in AI-assisted robotics programming.

This research could significantly contribute to the field by providing tools for more reliable implementation of LLMs in HRI applications and informing future developments in LLM training for spatial reasoning and robotic control tasks.

However, this study has limitations such as the use of a simplified "Robot class" with basic movements and integer coordinates does not fully capture the complexity of real-world Human-Robot Interaction (HRI) systems. This abstraction was employed intentionally to isolate and evaluate the core capabilities of the three models in a controlled, testable environment, without introducing the additional challenges posed by real-world robotic systems. The goal was to assess the models' basic functional performance before exploring more complex and dynamic scenarios.

Conclusion

This study has provided valuable insights into the capabilities of LLMs in generating code for HRI applications. Through a comparative analysis of ChatGPT 3.5, Gemini 1.5 Pro, and Claude 3.5 Sonnet, we have demonstrated that recent advancements in LLM technology have significantly improved their ability to understand and implement complex robotics programming tasks.

Our findings reveal a clear progression in performance across the three models, with Claude 3.5 Sonnet achieving a 95% success rate in generating accurate and efficient code for an abstract robot class. This high level of performance suggests that state-of-the-art LLMs are approaching a point where they could serve as valuable tools in robotics development processes, potentially accelerating prototyping and reducing initial coding efforts.

However, the study also highlights ongoing challenges, particularly in spatial reasoning and adherence to specific constraints, as evidenced by the lower success rates of Gemini 1.5 Pro and ChatGPT 3.5. These results underscore the complexity of translating natural language instructions into precise, error-free code for robotic operations and the need for continued refinement of LLM training in specialized domains.

In conclusion, we recognize that the simplifications of the current work might limit the generalizability of the findings to more advanced robotic systems. In future work, we aim to extend this study by incorporating more complex robot models that account for real-world challenges such as continuous motion, dynamic environments, and complex sensor data integration. Furthermore, we will explore existing literature on comparisons between code generation models in robotics and HRI to better frame our findings within the broader context of current research, highlighting both the novelty and limitations of our approach.

Disclosure Statement

No potential conflict of interest was reported by the author(s).

Funding

This research has received funding from the University of Brighton Research and Knowledge Exchange (RAKE) scheme.

ORCID

Nikolaos Polatidis http://orcid.org/0000-0003-4249-4953

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

References

Ahn, M., A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, and A. Zeng. 2022. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*.

Askell, A., Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma, et al. 2021. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*.

Austin, J., A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Ba¨rmann, L., R. Kartmann, F. Peller-Konrad, J. Niehues, A. Waibel, and T. Asfour. 2024. Incremental learning of humanoid robot behavior from natural interaction and large language models. *Frontiers in Robotics and AI* 11:1455375. doi: 10.3389/frobt.2024.1455375.

Bai, Y., A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al. 2022. Training a helpful and harm- less assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.

Bai, Y., S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.

Bommasani, R., D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

Breazeal, C., K. Dautenhahn, and T. Kanda. 2016. Social robotics. *Springer Handbook of Robotics* 1935–1972.

Brown, T. B. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.

Chen, M., J. Tworek, H. Jun, Q. Yuan, H. Ponde De Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Child, R., S. Gray, A. Radford, and I. Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*.

Hoffman, G., and X. Zhao. 2020. A primer for conducting experiments in human–robot interaction. *ACM Transactions on Human-Robot Interaction (THRI)* 10 (1):1–31. doi: 10.1145/3412374.

Husain, H., H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.

Izquierdo-Badiola, S., G. Canal, C. Rizzo, and G. Alenya`. 2024. PlanCollabNL: Leveraging large language models for adaptive plan generation in human-robot collaboration. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan, 17344–50, IEEE, May.

Kwon, T., N. Di Palo, and E. Johns. 2024. Language models as zero-shot trajectory generators. *IEEE Robotics and Automation Letters*, 9 (7): 6728–6735.

Lewis, P., P. Ethan, P. Aleksandra, P. Fabio, K. Vladimir, N. Goyal, H. Ku¨ttler, M. Lewis, W.-T. Yih, T. Rockta¨schel, et al. 2020. Retrieval- augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33:9459–74.

Li, Y., D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, K. James, G. Felix, D. L. Agustin, et al. 2022. Competition-level code generation with alphacode. *Science* 378 (6624):1092–97. doi: 10.1126/science.abq1158.

Mahadevan, K., J. Chien, N. Brown, Z. Xu, C. Parada, F. Xia, A. Zeng, L. Takayama, and D. Sadigh. 2024. Generative expressive robot behaviors using large language models. *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, Boulder, CO, USA, 482–91.

Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35:27730–44.

Park, S., C. C. Menassa, and V. R. Kamat. 2025. Integrating large language models with multimodal virtual reality interfaces to support collaborative human–robot construction work. *Journal of Computing in Civil Engineering* 39 (1):04024053. doi: 10.1061/JCCEE5.CPENG-6106.

Pearce, H., B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, 754–68, IEEE.

Perez, E., S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving. 2022. Red teaming language models with lan- guage models. *arXiv preprint arXiv:2202.03286*.

Radford, A., J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1 (8):9.

Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21 (140):1–67.

Sheridan, T. B. 2016. Human–robot interaction: Status and challenges. *Human Factors* 58 (4):525–32. doi: 10.1177/0018720816644364.

Smith, S., M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V. Korthikanti, et al. 2022. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*.

Song, C. H., J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, Paris, France, 2998–3009.

Vaswani, A. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*, Long Beach, California, USA, 6000–6010. doi:10.5555/3295222.3295349.

Wang, C., S. Hasler, D. Tanneberg, F. Ocker, F. Joublin, A. Ceravola, J. Deigmoeller, and M. Gienger. 2024. LaMI: Large language models for multi- modal human-robot interaction. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, Honolulu, HI, USA, 1–10.

Wei, J., Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

Winfield, A. F., and M. Jirotka. 2018. Ethical governance is essential to building trust in robotics and artificial intelligence systems. *Philosophical Transactions of the Royal Society*

A: Mathematical, Physical and Engineering Sciences 376 (2133):20180085. doi: 10.1098/rsta.2018.0085.

Xu, F. F., U. Alon, G. Neubig, and V. Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, San Diego, CA, USA, 1–10.

Zeng, F., W. Gan, Y. Wang, N. Liu, and P. S. Yu. 2023. Large language models for robotics: A survey. *arXiv preprint arXiv:2311.07226*.

Zhang, C., J. Chen, J. Li, Y. Peng, and Z. Mao. 2023. Large language models for human–robot interaction: A review. *Biomimetic Intelligence and Robotics* 3 (4):100131. doi: 10.1016/j.birob.2023.100131.